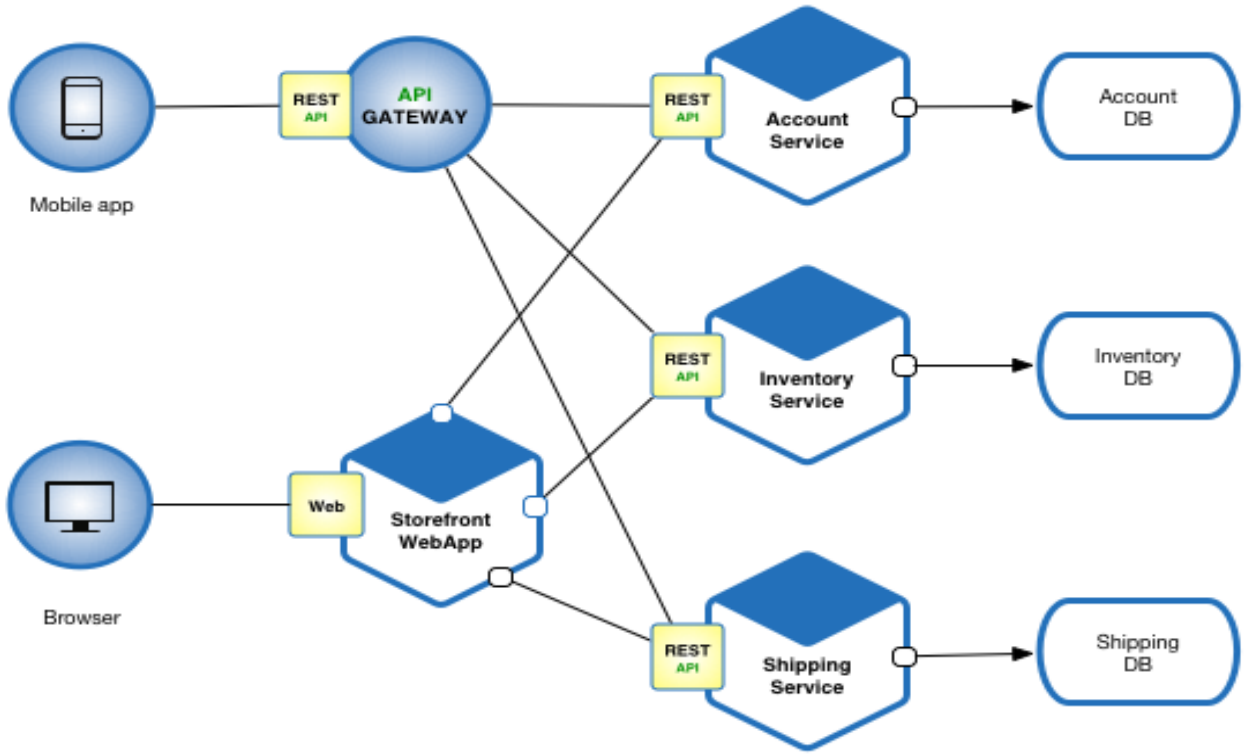
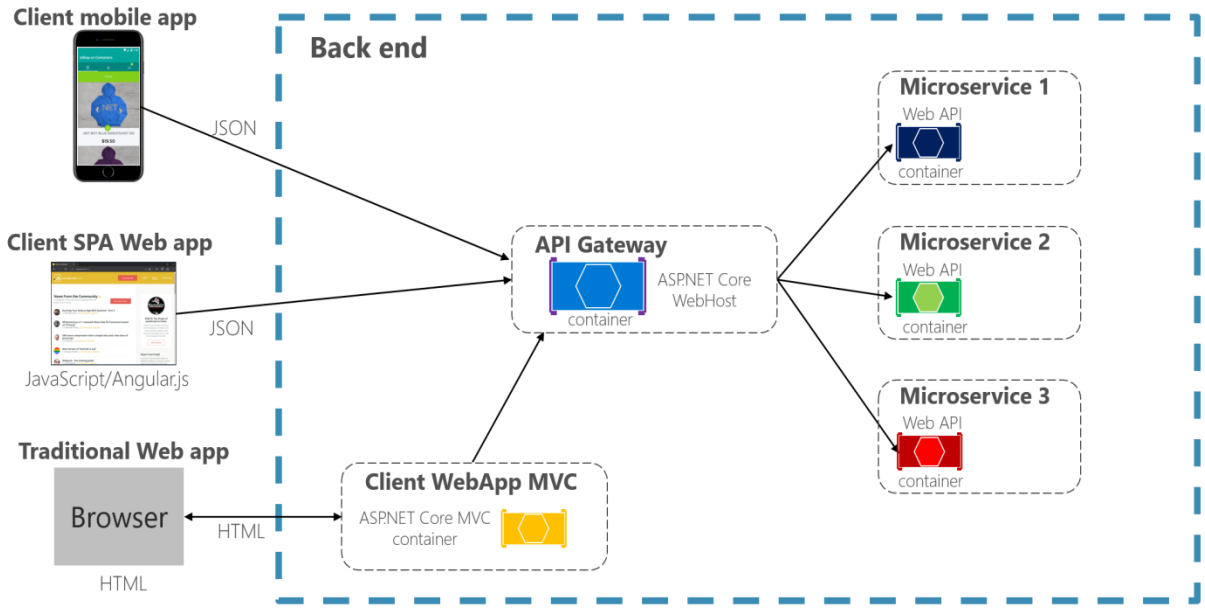
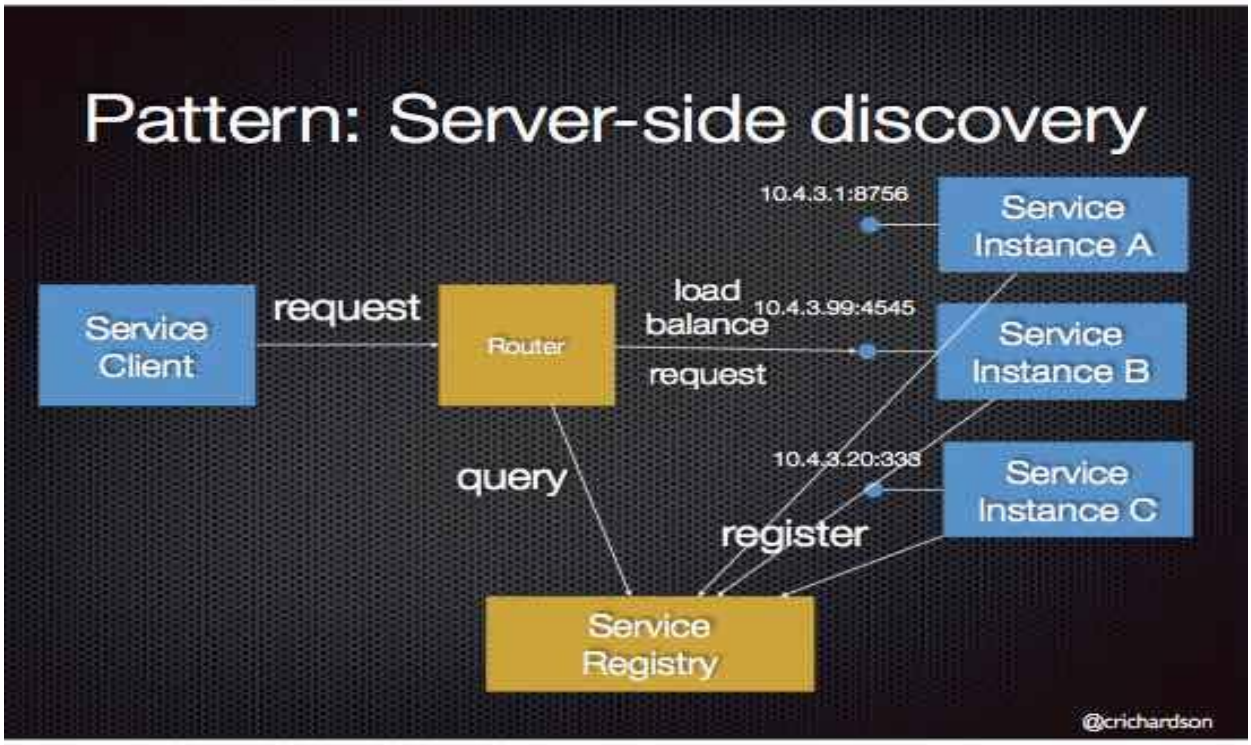
Below is a **comprehensive, industry-standard master list of Microservice Design Patterns**, grouped by **category**, with **clear purpose and real-world usage**. This is suitable for **system design interviews, architecture documents, and production systems**.




Using a single custom **API Gateway service**






---

**1 Decomposition & Service Design Patterns**

*(How to split and structure microservices)*

| Pattern | Purpose |
|---|---|
| **Decompose by Business Capability** | Services aligned with business domains |
| **Decompose by Subdomain (DDD)** | Based on bounded contexts |
| **Single Responsibility Service** | One service = one responsibility |
| **Self-Contained Service** | Independent UI, logic, data |
| **Database per Service** | Avoid shared databases |
| **Shared Database (Anti-Pattern)** | Tight coupling (avoid) |
| **Stateless Service** | Enables easy scaling |
| **Service Instance per Container** | One service per container |

---

## 2 API & Communication Patterns

*(How services talk to each other)*

| Pattern | Purpose |
|---|---|
| **API Gateway** | Single entry point for clients |
| **Backend for Frontend (BFF)** | Client-specific APIs |
| **Aggregator** | Combines responses from multiple services |
| **Fan-Out** | One request → multiple services |
| **Chained Service Calls** | Sequential service calls |
| **Asynchronous Messaging** | Loose coupling via events |
| **Publish–Subscribe** | Event broadcasting |
| **Request–Reply** | Synchronous communication |
| **Message Broker** | Kafka / RabbitMQ based messaging |

---

## 3 Data Management Patterns

*(Handling data consistency & transactions)*

| Pattern | Purpose |
|---|---|
| **Database per Service** | Strong isolation |
| **Saga** | Distributed transaction handling |
| **Saga (Choreography)** | Event-driven coordination |
| **Saga (Orchestration)** | Central coordinator |
| **Event Sourcing** | Store state as events |
| **CQRS** | Separate read/write models |
| **API Composition** | Aggregate data at API layer |
| **Materialized View** | Pre-computed read models |
| **Two-Phase Commit (Anti-Pattern)** | Poor scalability |

---

**4 Resilience & Fault Tolerance Patterns**

*(Survive failures gracefully)*

| Pattern | Purpose |
|---|---|
| **Circuit Breaker** | Prevent cascading failures |
| **Retry** | Handle transient failures |
| **Timeout** | Avoid long waits |
| **Bulkhead** | Isolate failures |
| **Fallback** | Default response on failure |
| **Rate Limiting** | Protect services |
| **Throttling** | Control traffic spikes |
| **Fail-Fast** | Immediate failure detection |
| **Graceful Degradation** | Partial functionality |

---

**5 Service Discovery & Networking Patterns**

*(Dynamic service location)*

| Pattern | Purpose |
|---|---|
| **Client-Side Discovery** | Client finds service |
| **Server-Side Discovery** | Load balancer finds service |
| **Service Registry** | Central service directory |
| **DNS-Based Discovery** | DNS resolution |
| **Sidecar** | Networking & security proxy |
| **Service Mesh** | Observability & traffic control |
| **Load Balancer** | Distribute traffic |

## 6 Configuration & Deployment Patterns

*(Environment & deployment control)*

| Pattern | Purpose |
|---|---|
| **Externalized Configuration** | Config outside code |
| **Config Server** | Centralized config |
| **Blue-Green Deployment** | Zero downtime |
| **Canary Deployment** | Gradual rollout |
| **Rolling Deployment** | Incremental update |
| **Feature Toggle** | Enable/disable features |
| **Immutable Infrastructure** | No in-place changes |
| **Container per Service** | Docker-based deployment |

## 7 Security Patterns

*(Protect microservices)*

| Pattern | Purpose |
|---|---|
| **API Gateway Security** | Central auth |
| **OAuth2 / JWT** | Token-based auth |
| **Mutual TLS (mTLS)** | Service-to-service security |
| **Zero Trust Architecture** | No implicit trust |
| **Secrets Management** | Secure credentials |
| **Token Relay** | Forward auth tokens |
| **Identity Provider** | Central identity management |

## 8 Observability & Monitoring Patterns

*(Visibility & debugging)*

| Pattern | Purpose |
|---|---|
| **Centralized Logging** | Unified logs |
| **Distributed Tracing** | Request flow tracking |
| **Correlation ID** | Trace request across services |
| **Health Check API** | Service health |
| **Metrics Aggregation** | Performance metrics |
| **Alerting** | Failure notifications |
| **Log Aggregator** | ELK / EFK stack |

---

## 9 Integration & Legacy Patterns

*(Interoperability & migration)*

| Pattern | Purpose |
|---|---|
| **Strangler Fig** | Gradual legacy replacement |
| **Anti-Corruption Layer** | Protect domain model |
| **Adapter** | Interface compatibility |
| **Façade** | Simplified access |
| **Sidecar Adapter** | Protocol translation |
| **Legacy Gateway** | Legacy system access |

---

### □ Anti-Patterns (Avoid These)

*(Common mistakes)*

| Anti-Pattern | Problem |
|---|---|
| **Distributed Monolith** | Tight coupling |
| **Chatty Services** | Excessive calls |
| **Shared Database** | Data coupling |
| **Hard-Coded Configuration** | Inflexible |
| **God Service** | Too much responsibility |
| **Synchronous Everything** | Poor resilience |

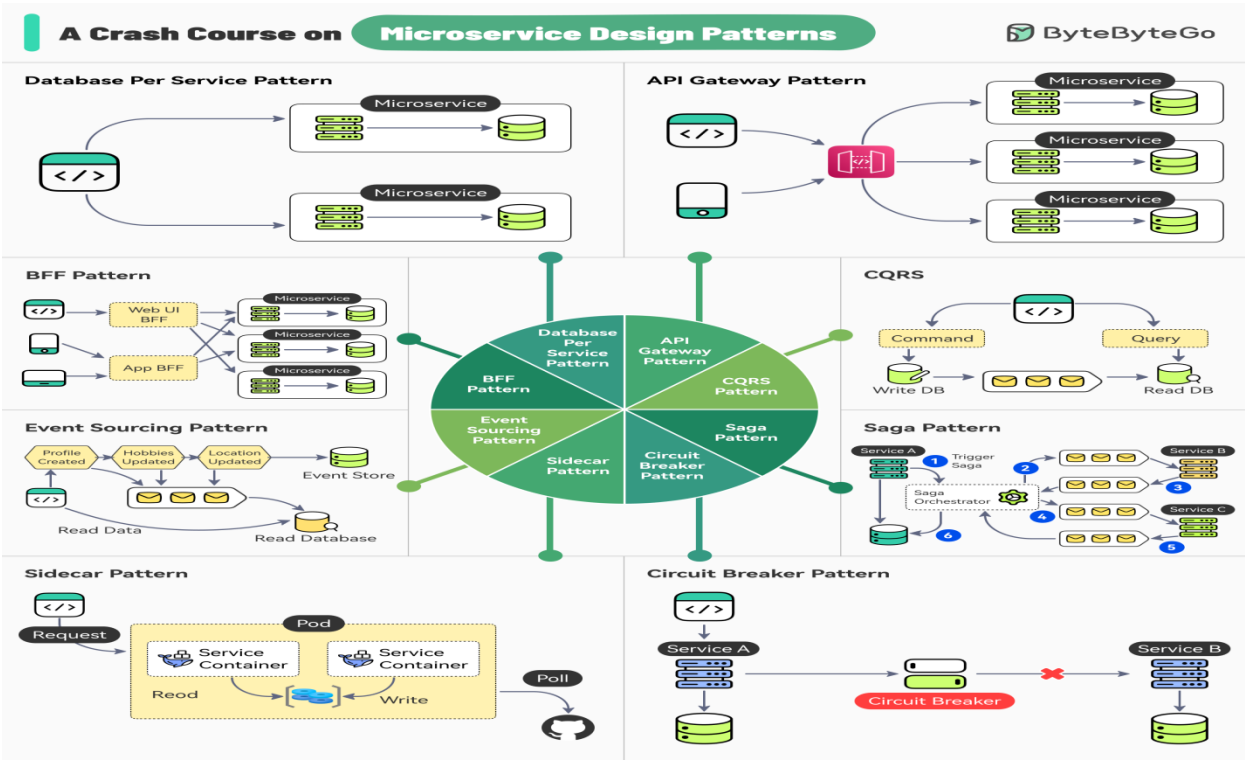
---

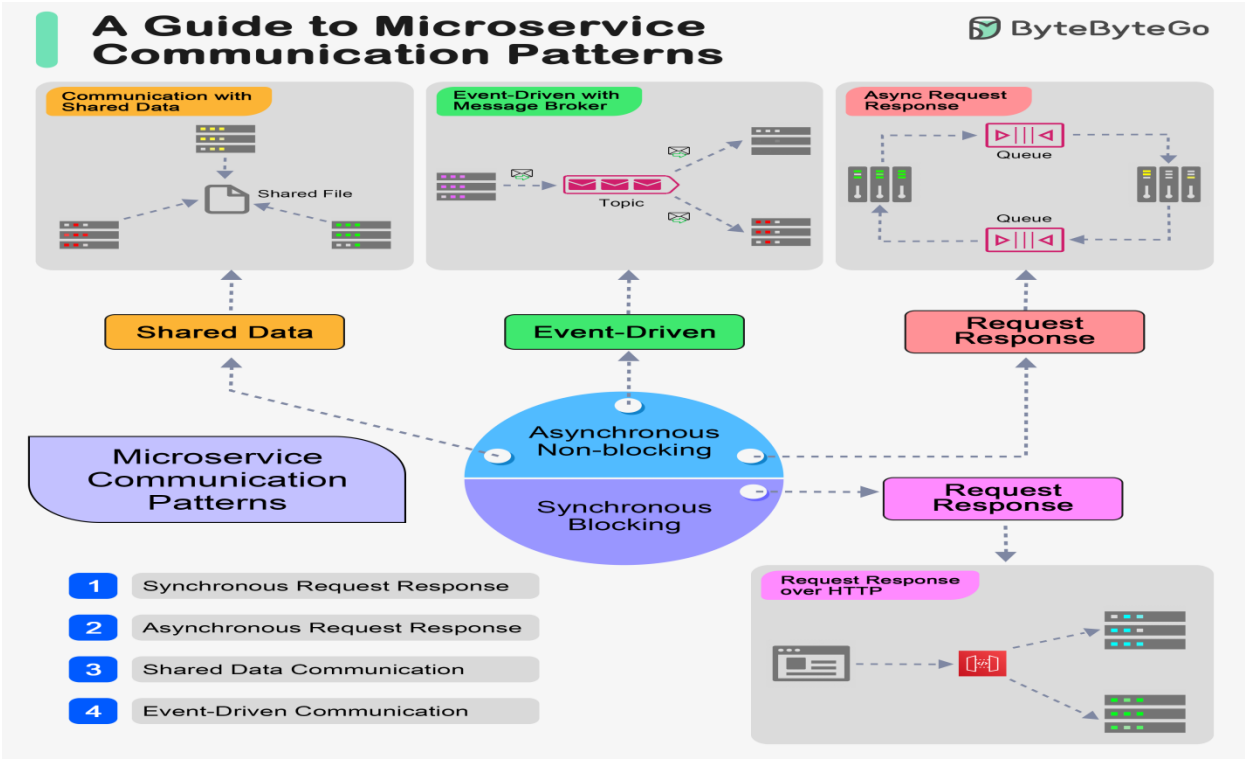
### □ Summary (Interview-Ready)

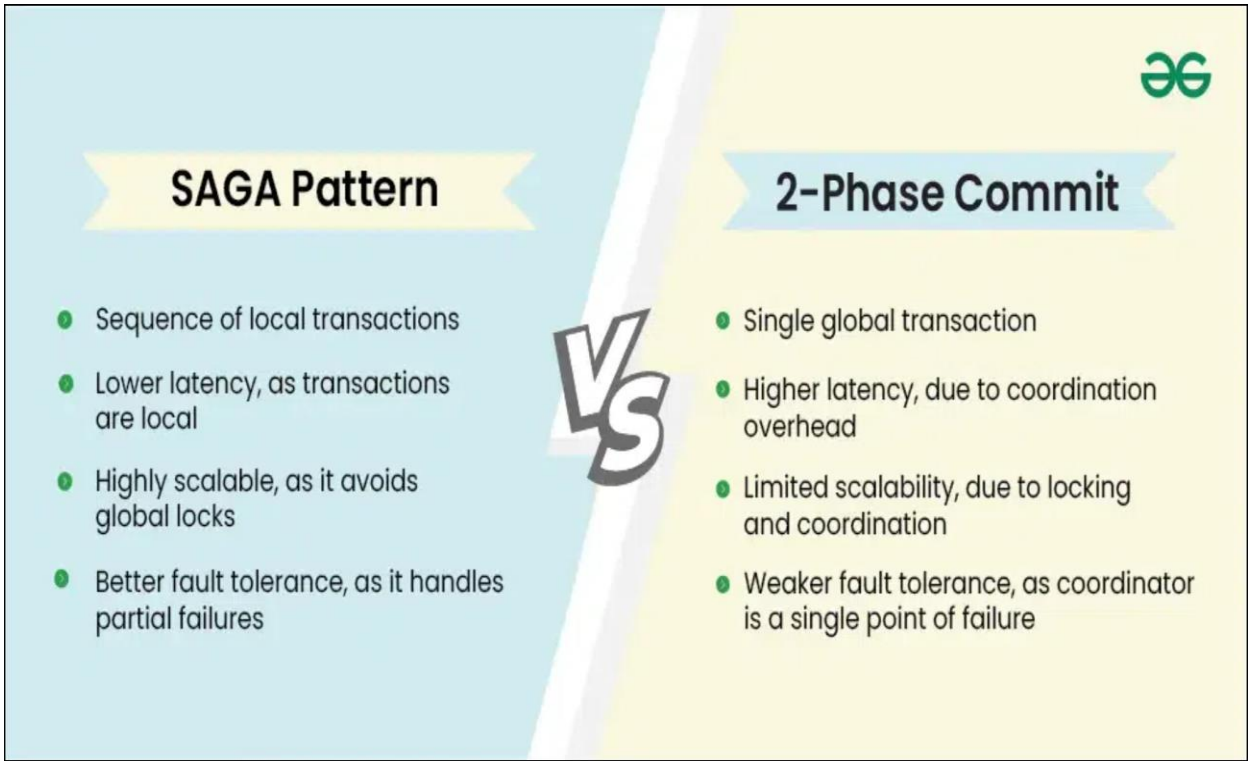
- **Core patterns**: API Gateway, Saga, Circuit Breaker, CQRS
- **Scalability**: Stateless services, async messaging
- **Resilience**: Bulkhead, Retry, Timeout
- **Security**: OAuth2, mTLS, Zero Trust
- **Observability**: Logging, tracing, metrics

---

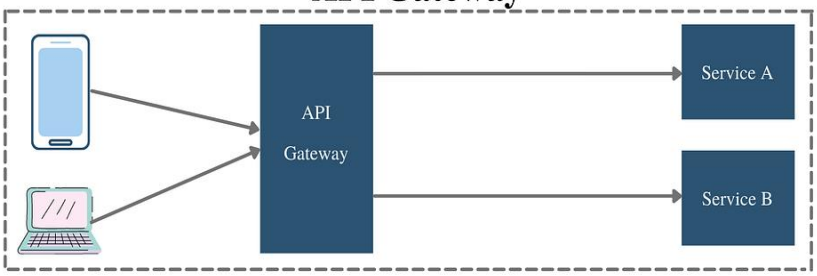
Below is a **single master comparison table** that clearly answers **"WHEN to use WHICH microservice design pattern"**.
This table is **architecture-review ready**, **interview-ready**, and **production-oriented**.



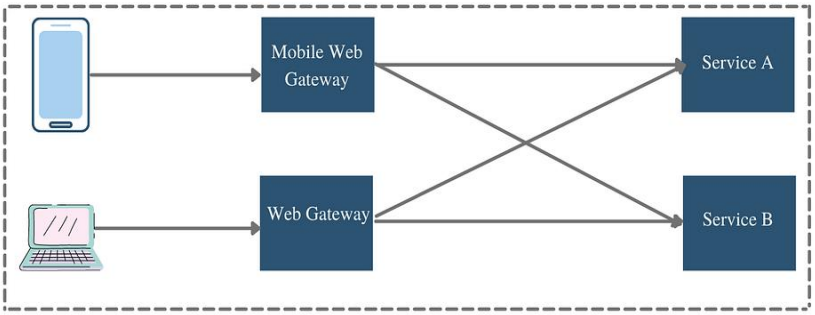



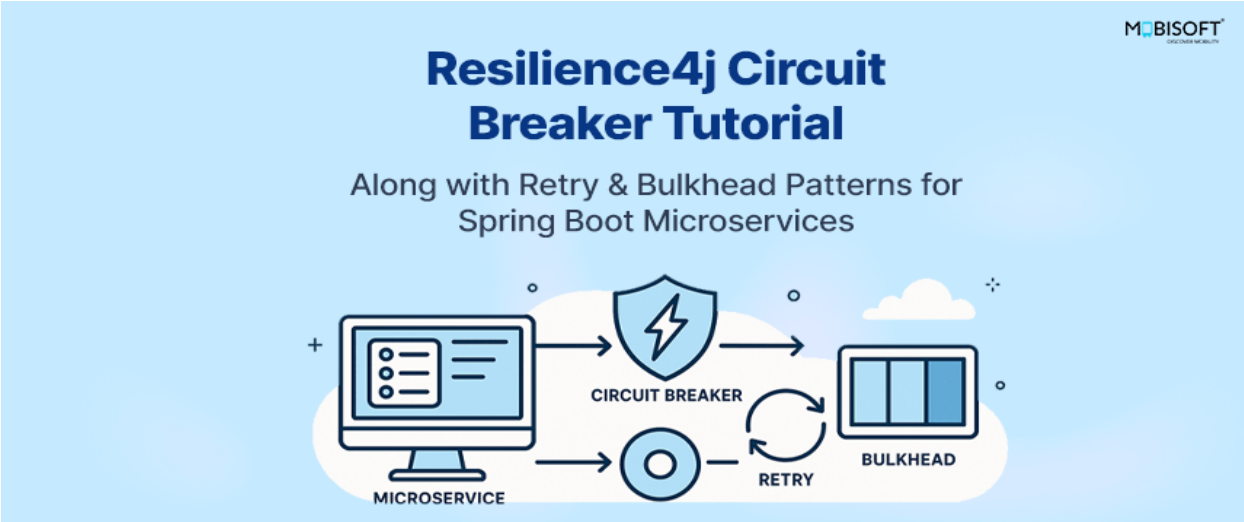



**API Gateway**




**Backend For Frontend**






---

 **One Master Comparison Table – *When to Use Which Microservice Pattern***

| Scenario / Problem | Use This Pattern | Why This Pattern Fits Best | Avoid This |
|---|---|---|---|
| **Breaking a monolith into services** | Decompose by Business Capability | Aligns services with business domains | Layer-based decomposition |
| **Clear domain boundaries exist** | Decompose by Subdomain (DDD) | Prevents cross-domain coupling | Shared domain models |
| **Multiple clients (Web, Mobile, IoT)** | Backend for Frontend (BFF) | Client-optimized APIs | One-size-fits-all API |
| **Single entry point needed** | API Gateway | Central auth, routing, throttling | Client-to-service calls |
| **Need to aggregate multiple services** | API Composition / Aggregator | Reduces client complexity | Chatty clients |
| **Services must scale independently** | Stateless Service | Horizontal scalability | Stateful services |
| **Avoid database coupling** | Database per Service | Independent evolution | Shared database |
| **Distributed transaction required** | Saga (Choreography) | Event-driven, scalable | Two-phase commit |

| | | | |
|---|---|---|---|
| **Complex workflow coordination** | Saga (Orchestration) | Central control of flow | Hard-coded service chains |
| **High read, low write workloads** | CQRS | Optimized read models | Single data model |
| **Need full audit/history** | Event Sourcing | Complete state history | Overwriting data |
| **Temporary network failures** | Retry | Handles transient issues | Infinite retries |
| **Prevent cascading failures** | Circuit Breaker | Stops failure propagation | Blind retries |
| **Slow or unreliable services** | Timeout | Frees resources quickly | Long blocking calls |
| **Failure isolation needed** | Bulkhead | Limits blast radius | Shared thread pools |
| **Graceful degradation required** | Fallback | Improves user experience | Hard failures |
| **Dynamic service instances** | Service Discovery | Handles scaling & failures | Hardcoded IPs |
| **Kubernetes / Cloud-native networking** | Service Mesh | Observability + security | Custom network code |
| **Zero downtime deployment** | Blue-Green Deployment | Safe switch-over | Big-bang release |
| **Gradual feature rollout** | Canary Deployment | Limits production risk | Full rollout |
| **Config varies per environment** | Externalized Configuration | No redeploy required | Hardcoded config |
| **Secure service-to-service calls** | mTLS | Strong authentication | Plain HTTP |
| **Centralized authentication** | OAuth2 / JWT | Standard, scalable | Custom auth logic |
| **Track request across services** | Distributed Tracing | Faster debugging | Manual log correlation |
| **Monitor service health** | Health Check Pattern | Automated recovery | Manual checks |

| | | | |
|---|---|---|---|
| **Replace legacy system gradually** | Strangler Fig | Safe modernization | Rewrite everything |
| **Protect domain from legacy model** | Anti-Corruption Layer | Clean domain design | Direct legacy access |
| **Integrate incompatible interfaces** | Adapter Pattern | Compatibility layer | Direct dependency |
| **Excessive synchronous calls** | Async Messaging | Loose coupling | Chatty REST |
| **Event broadcasting needed** | Publish–Subscribe | Decoupled communication | Point-to-point calls |

---

### □ Quick Decision Guide (Cheat Sheet)

| **If your main concern is…** | **Go with…** |
|---|---|
| **Scalability** | Stateless Service, Async Messaging |
| **Reliability** | Circuit Breaker, Retry, Bulkhead |
| **Data consistency** | Saga, CQRS |
| **Performance** | API Gateway, Aggregator |
| **Security** | OAuth2, mTLS |
| **Observability** | Distributed Tracing, Centralized Logging |
| **Deployment safety** | Blue-Green, Canary |
| **Legacy migration** | Strangler Fig |

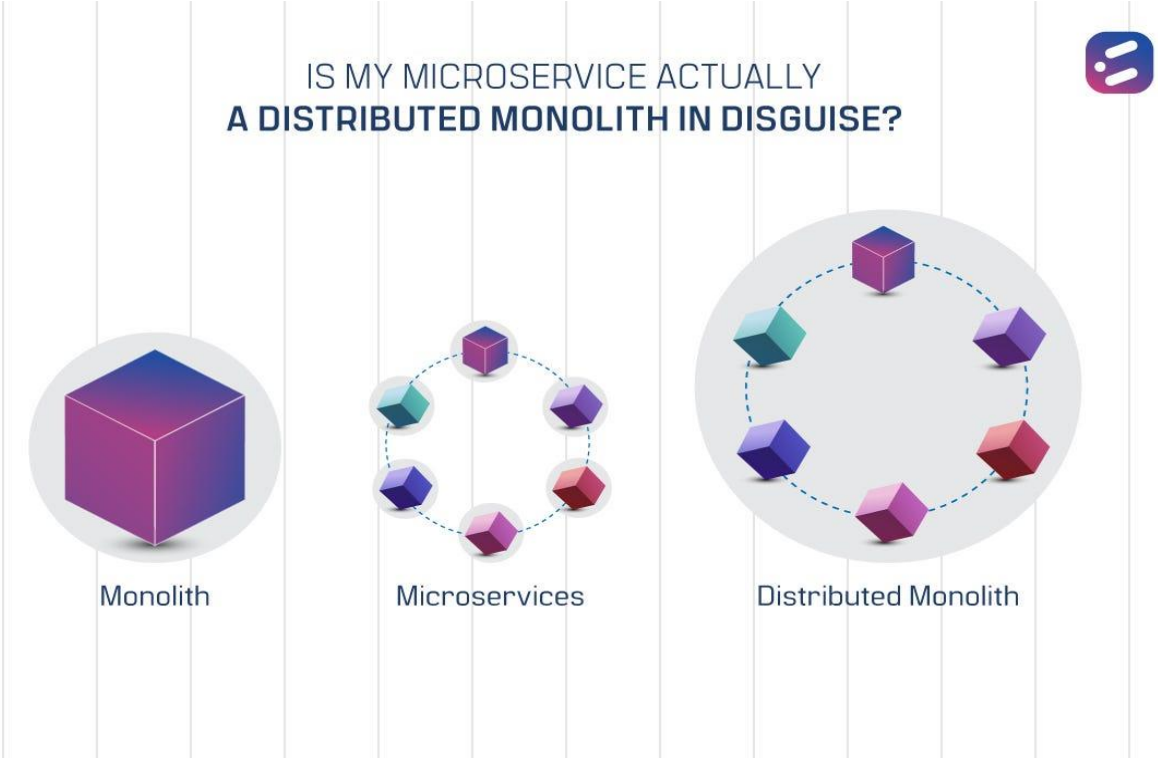
---

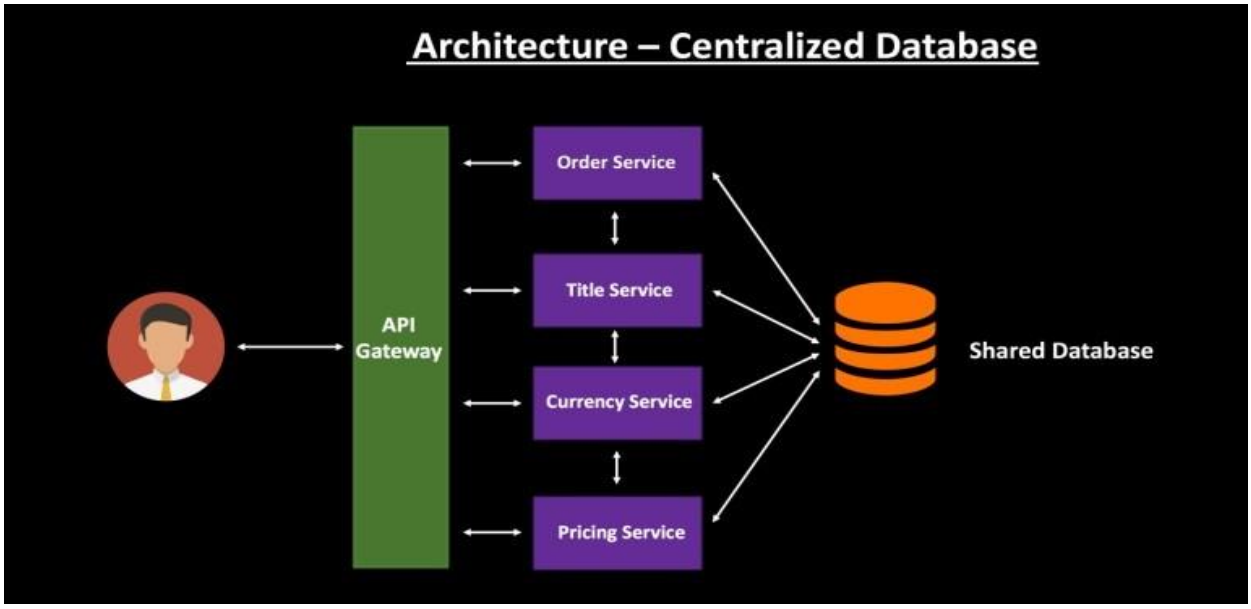
### □ Interview Tip (Use This Line)

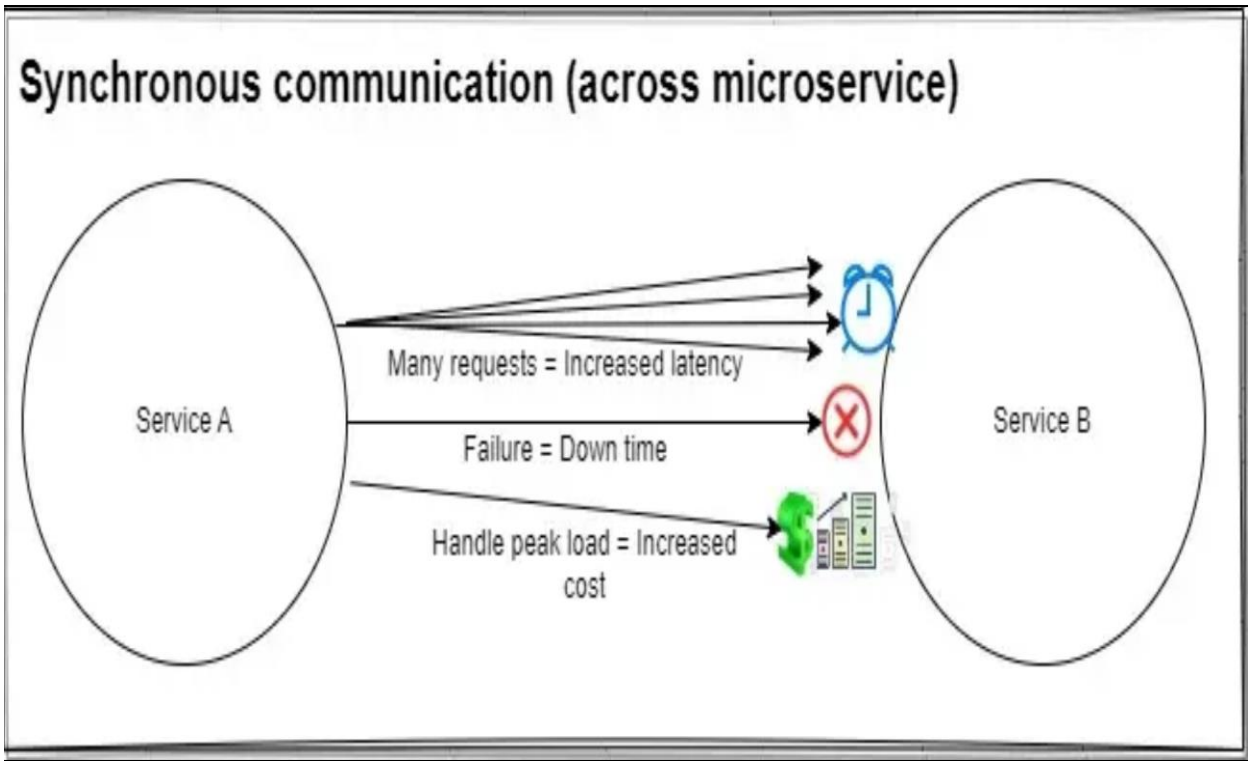
*"In microservices, pattern selection depends on business boundaries, failure tolerance, data consistency requirements, and deployment strategy—not technology alone."*

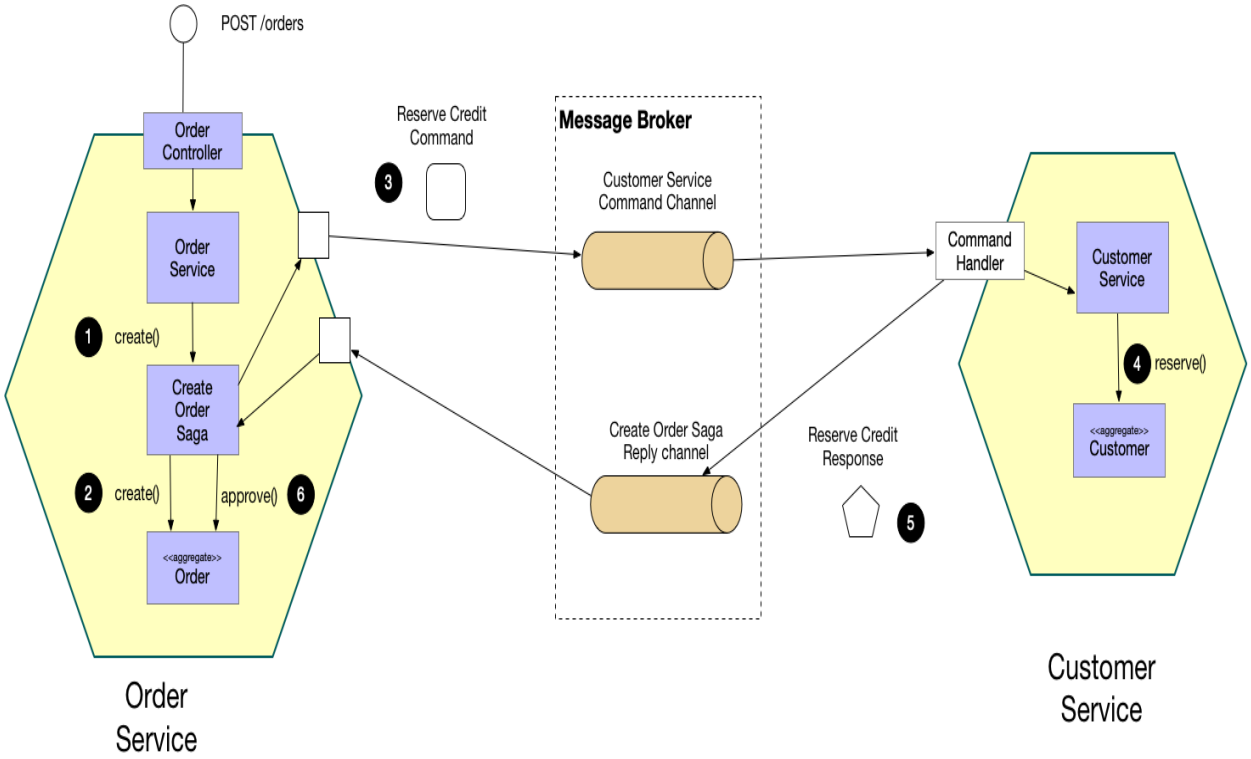
---

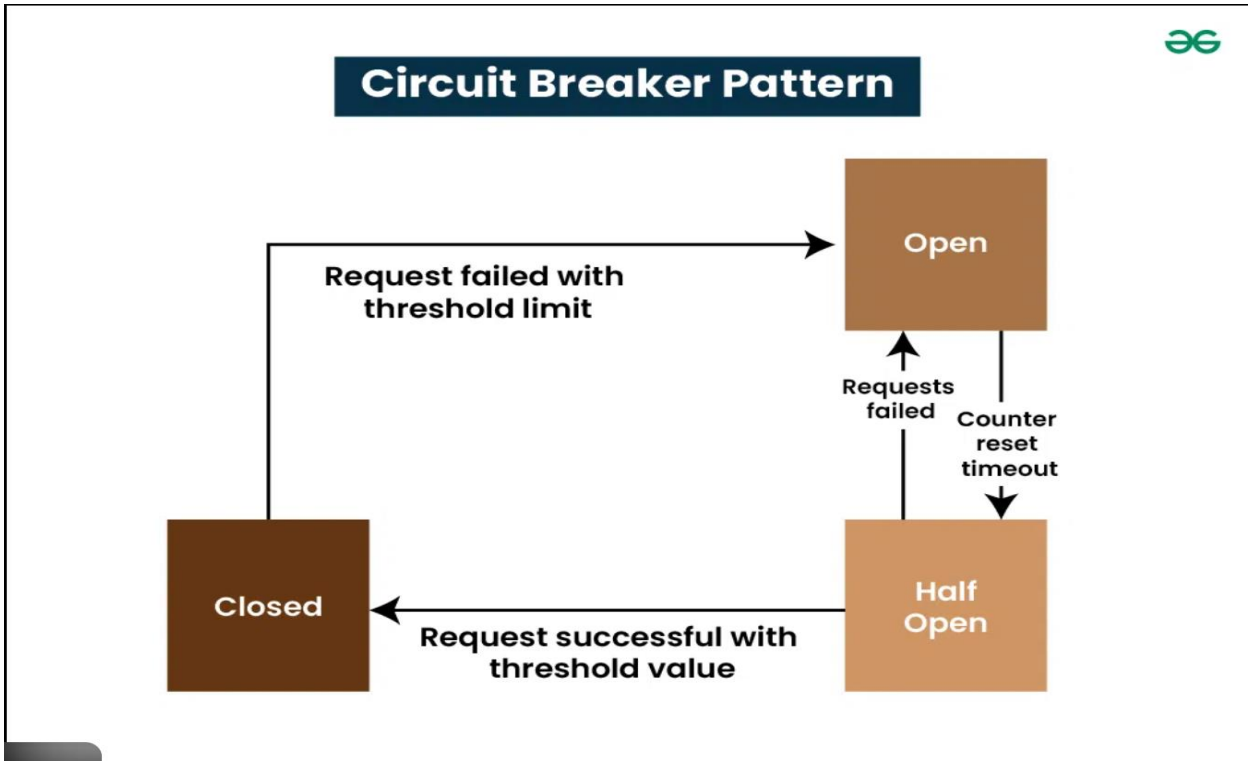
Below is a **clean, interview-ready, production-oriented Anti-Pattern → Correct-Pattern mapping** for **Microservices Architecture**.
This table is commonly used in **architecture reviews, senior-level interviews, and design documents**.




Synchronous communication (across microservice)
Service A
Many requests = Increased latency
Failure = Down time
Handle peak load = Increased cost
Service B


POST /orders
Order Controller
Order Service
1 create()
Create Order Saga
2 create()
approve() 6
<<aggregate>> Order
Order Service
Reserve Credit Command
3
Message Broker
Customer Service Command Channel
Create Order Saga Reply channel
Reserve Credit Response
5
Command Handler
Customer Service
4 reserve()
<<aggregate>> Customer
Customer Service




---

### ❌ Anti-Pattern → ✅ Correct Pattern Mapping (Master Table)

| ❌ Anti-Pattern | Why It Fails | ✅ Correct Pattern | How It Fixes the Problem |
|---|---|---|---|
| **Distributed Monolith** | Tight coupling across services | Decompose by Business Capability | Independent evolution |
| **Shared Database** | Schema & deployment coupling | Database per Service | Full data isolation |
| **Chatty Services** | High latency & failure risk | API Composition / Aggregator | Fewer network calls |
| **Synchronous Everything** | Poor resilience & scaling | Async Messaging | Loose coupling |
| **God Service** | Single point of failure | Single Responsibility Service | Clear ownership |
| **Hardcoded Configuration** | Environment inflexible | Externalized Configuration | Runtime flexibility |

| | | | |
|---|---|---|---|
| **Tight Domain Coupling** | Legacy leaks into domain | Anti-Corruption Layer | Domain protection |
| **Two-Phase Commit** | Poor scalability | Saga Pattern | Eventual consistency |
| **Direct Service Calls** | Fragile dependency graph | API Gateway | Centralized access |
| **No Failure Isolation** | Cascading failures | Bulkhead Pattern | Blast-radius control |
| **Blind Retries** | Retry storms | Retry + Circuit Breaker | Controlled retries |
| **Long Blocking Calls** | Resource exhaustion | Timeout Pattern | Fast failure |
| **Manual Load Balancing** | Not cloud-native | Service Discovery | Dynamic routing |
| **Client Knows Services** | Tight client coupling | API Gateway / BFF | Client abstraction |
| **Large Deployment Units** | Slow releases | Independent Deployments | Faster iteration |
| **No Observability** | Debugging nightmare | Distributed Tracing | End-to-end visibility |
| **Log Files per Service** | Hard troubleshooting | Centralized Logging | Unified logs |
| **No Health Checks** | Undetected failures | Health Check Pattern | Auto-recovery |
| **Version Lockstep** | Forced upgrades | Backward-Compatible APIs | Independent evolution |
| **Over-Fetching Data** | Performance loss | CQRS | Optimized reads |
| **Shared Auth Logic** | Security duplication | Central Auth (OAuth2/JWT) | Standard security |
| **Hard Dependencies on Legacy** | High migration risk | Strangler Fig | Gradual replacement |
| **Excessive REST Calls** | High latency | Event-Driven Architecture | Asynchronous flow |

| | | | |
|---|---|---|---|
| **Static IP Usage** | Scaling issues | Service Registry | Dynamic resolution |
| **Security at Each Service** | Inconsistent policies | mTLS / Service Mesh | Uniform security |
| **Big-Bang Release** | High production risk | Canary / Blue-Green | Safer releases |

---

### □ Pattern Selection Logic (Why Architects Care)

| Problem Category | Anti-Pattern Symptom | Correct Pattern Focus |
|---|---|---|
| **Scalability** | Shared state | Stateless services |
| **Reliability** | Cascading failures | Circuit breaker |
| **Data** | Strong consistency | Saga |
| **Performance** | Too many calls | Aggregation |
| **Security** | Inconsistent auth | Central auth |
| **Observability** | No tracing | Distributed tracing |
| **Deployment** | Risky releases | Canary / Blue-Green |

---

### □ Interview One-Liners (High-Impact)

- **"Shared database turns microservices into a distributed monolith."**
- **"If everything is synchronous, nothing is resilient."**
- **"Two-phase commit kills scalability; Saga restores it."**
- **"Without observability, microservices are unmanageable."**

---

### □ Architect's Golden Rule

*Every microservice anti-pattern exists because a monolith rule was blindly copied into a distributed system.*